

Hexagon Bridge

Make the Hexagon NPU reachable for models nobody converted for you — and prove, on the laptop itself, that the NPU is really what ran them.

Every number measured

A 45 TOPS NPU you cannot reach

Snapdragon X Elite, HP OmniBook — for any model outside a vendor catalog

1

The standard recipe quietly costs you

`quantize_dynamic()` is what every tutorial shows. On a real X Elite it cost MobileNetV2 72 points of accuracy — and crashed the runtime outright on whisper-base.

2

ONNX Runtime hands you a CPU session

The documented `providers=[...]` argument silently yields CPU-only while your logs say NPU.

3

The chip rejects what every local check passed

One sign bit on one input of one op type: nine LayerNorms refused on silicon, after a clean local run.

Every one of these failures is silent. The app works. The battery doesn't.

Four parts, one question

Will this model run on the Hexagon NPU — and did it?

Scanner

Answers it on any Windows PC with no Snapdragon device: static rules first, then Qualcomm's own HTP compiler locally in about 4 seconds. Names the ops and the reason.

Converter

Static a16w8 QDQ the QNN-specific way, calibrated on real data — including the fixes that are not in any quickstart.

Validator

Runs the model on a real Snapdragon X Elite and reconciles the prediction against what the chip actually did.

App

Speech-to-text server and dashboard that report the execution provider that attached, never the one requested.

Built on what already exists: ONNX Runtime's strict no-fallback session, Olive's attention-mask surgery, Qualcomm AI Hub. The missing piece was diagnosis — which failure you have, and which known fix applies.

Measured on a real Snapdragon X Elite

Qualcomm AI Hub · SC8380XP, Hexagon v73 · every number committed as evidence in models/reports/

17.8 ms

median encoder latency, 100 runs

185 / 187

layers on the NPU — the other two are
graph-edge conversions

12.70%

word error rate — identical to the FP32
reference

5.2 → 0.5 s

cold vs cached graph load

The silent fallback, priced

A quantized model that misses the NPU runs 26–28% slower than not quantizing at all — measured in two separate sessions on the same device. Quantizing without checking placement is worse than doing nothing.

And the scanner catches what the chip rejects — before you own the chip

The exact model a real X Elite refused is flagged two ways locally: statically, and by running Qualcomm's compiler. Both device logs — the refusal and the acceptance — are committed in the repository.

Four traps, stacked

Each one passes every check before it

“Then don't quantize”

The HTP runs float as FP16, and the unquantized encoder compiles into one NPU graph locally. On the device: failed to finalize, error 6000. Twice.

Local SDK 2.50, device 2.45.

The wrong format

Dynamic quantization doesn't reliably cost you the NPU — it reliably costs accuracy. MobileNetV2: 79.5% → 7.2% top-1, with perfect placement.

whisper-base crashed the runtime.

The wrong precision

8-bit activations scan 100% NPU-eligible, compile to one graph, and transcribe gibberish: 93.5% word error rate. 16-bit fixes it.

Placement says nothing about answers.

Only the chip knows

a16w8 scanned clean, measured cosine 0.99 locally — and the X Elite refused all nine LayerNorms because gamma was signed int8.

Unsigned weights: accepted.

Trap 2 was my own claim, corrected: the scanner used to say dynamic quantization means 0% NPU. The device disagreed, so the rule changed.

It isn't one model

Five popular models, both paths, each profiled on the same X Elite

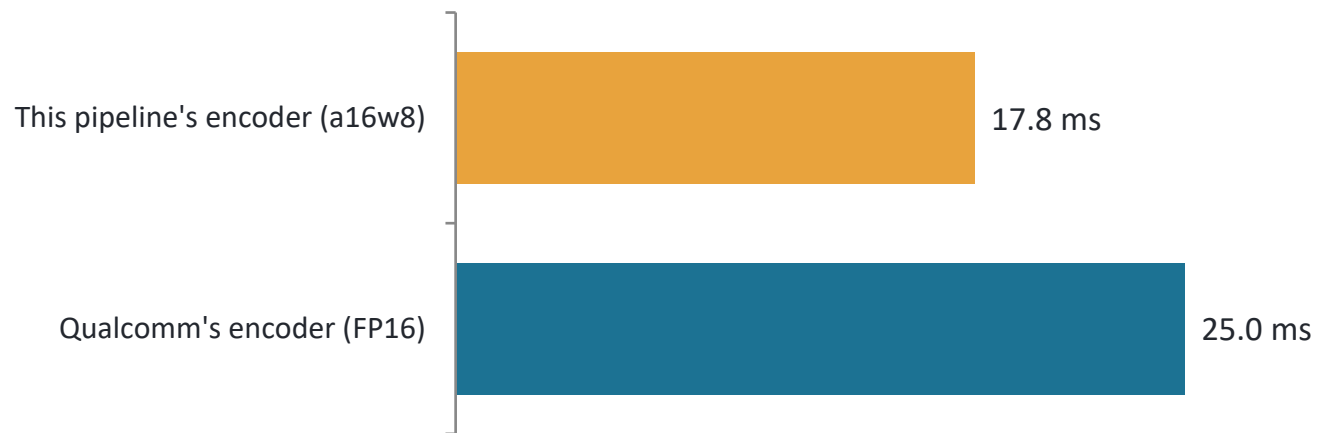
Model	What it's for	FP32 → naive → this pipeline	On the NPU
MobileNetV2	Image classification	79.5% → 7.2% → 79.5% top-1	0.36 ms
Whisper-base	Speech recognition	8.84% → 11.33% → 8.84% WER	45.7 ms
MiniLM-L6	Embeddings for local search	100% → 63.3% → 75.0% same top hit	1.29 ms
DistilBERT SST-2	Sentiment	90.7% → 90.7% → 91.0%	2.44 ms
CLIP ViT-B/32	Image search	98.2% → 98.8% → 98.2% zero-shot	2.55 ms

The exporter mattered more than the quantizer

PyTorch's newer exporter decomposes attention into a graph carrying GatherND and IsNaN. That split every transformer here into two NPU graphs, and on the real device they then failed to allocate memory at all. Exporting with the TorchScript exporter and eager attention removed those ops — the same models then compiled into one graph and ran. The model was never the problem.

Against Qualcomm's own NPU Whisper

Their pre-compiled X Elite build, profiled by me on the same device, through the same harness



2.60 ms

per token — Qualcomm's decoder, 509 of 509 layers on the NPU

The piece this project does not yet run on the NPU. Now measured, not assumed.

Read it honestly

- The two models are not the same graph — Qualcomm restructures attention and stays in float, this pipeline quantizes.
- The runs are eight days apart on a shared device pool.
- So: a general-purpose quantization path can land in the same league as the vendor's hand-optimised build. Not a benchmark win.

One command on a Snapdragon HP laptop

Because “it uses the NPU” should be something you can check, not something you're told

```
powershell -ExecutionPolicy Bypass -File .\setup-snapdragon.ps1
```

1

ARM64-native Python

x64 Python under emulation can never load QnnHtp.dll — the NPU is unreachable and nothing says so.

2

The NPU is visible

Registers the plugin properly and reports the devices ONNX Runtime really sees. Names the driver version if it's too old.

3

The model is on it

Builds a session with CPU fallback disabled. If anything would fall back, the session refuses to start.

No Snapdragon hardware needed for the pre-flight check itself — the scanner and the HTP compile check run on any Windows PC.

What I did not prove

The same document that carries the wins carries these

Only the encoder is on the NPU

The decoder still runs on CPU here. The speedups quoted are encoder speedups. Qualcomm's decoder measurement shows it can live on the NPU; wiring it in needs a physical device to verify.

Speedup depends on the CPU baseline

24–57× against AI Hub's CPU numbers, but 4–10× against a well-tuned CPU. The honest figure needs a physical X Elite.

No power or battery measurement

AI Hub doesn't expose it. The efficiency argument for an NPU is the one thing I could not measure.

MiniLM loses retrieval quality

75.0% same top hit vs FP32. Embedding cosine of 0.987 looks fine and hides it — a16w8 is not free for every model.

Make the NPU checkable

The Hexagon NPU is the reason to buy the laptop. Today it's reachable only for models someone else already converted — and when it isn't reached, nothing tells you. This turns that into a claim you can check for your model, on your machine, in seconds.

6 models

converted and run on a real X Elite

31 tests

each pinning a bug that once gave a wrong number

Every claim

traceable to a committed evidence file